

Complete Master Plan

How every channel connects · How barcodes are generated · How the full daily workflow runs — explained step by step, from scratch to live system.

DOCUMENT TYPE	Detailed Technical & Workflow Master Plan
AUDIENCE	Team Leads · Developers · Management
MODULES COVERED	10 Core Modules · End-to-End Flow
CHANNELS	Website · Mobile App · WhatsApp · Manual ERP
DATE	22 May 2026
VERSION	1.0 — Architecture Phase

Table of Contents

Everything covered in this document, in order.

PART 1	The Big Picture	What are we building and why?
PART 2	The Four Order Channels	Website · App · WhatsApp · Manual
PART 3	Central Order Hub	How all orders land in one place
PART 4	Hub Assignment Engine	How the right hub is chosen automatically
PART 5	FF GM Command Dashboard	What the manager sees and controls
PART 6	EOD PO Engine	How Purchase Orders are auto-generated at 11:59 PM
PART 7	Purchase Executive Portal	How the PO reaches the right person
PART 8	Barcode & QR Generation	What a barcode is, what goes in it, how it prints
PART 9	Hub Manager Scanner App	React Native app — scan, receive, QC, wastage
PART 10	Smart Inventory Management	How stock levels stay always accurate
PART 11	Delivery & Distribution	Pack, scan, dispatch, confirm
PART 12	PO Bill & Approval Chain	Auto bill → GM → Admin → CEO → Payment
PART 13	Technology Stack Explained	Every tool, explained simply
PART 14	Database Tables Overview	The data behind every step
PART 15	Implementation Phases	What to build first, second, third

The Big Picture

What are we building and how does it all fit together?

What Is the ERP?

ERP stands for Enterprise Resource Planning. Think of it as the brain of your entire business. Every department — Sales, Purchase, Warehouse, HR, Finance — talks to this one system. Instead of using WhatsApp here, Excel there, and phone calls everywhere, everything happens in one place. That is the IGO Group ERP.

■ **SIMPLE ANALOGY — What is an ERP?**

Imagine your business is a school. The ERP is like the school office — every teacher, student, and parent goes through it. The office knows who is absent, who paid fees, what books are needed. Without it, everyone would be calling each other on the phone and things would get lost. The ERP is that central office for your business.

What Are We Building Specifically?

We are building the Order-to-Delivery pipeline inside the ERP. This means: a customer places an order anywhere (website, app, WhatsApp, phone call) — and without anyone manually doing anything, the system figures out which hub to send from, creates a purchase order to restock, sends it to the right person, barcodes the boxes, updates inventory when they arrive, dispatches delivery, and processes the vendor payment. All automatic. All in one system.

The Daily Cycle — Overview

Time	Event	Details
10:00 AM	Order collection starts	BD team + website + app + WhatsApp all active
All Day	Orders accumulate	Every order tagged with hub, creator, time, source
Every Hour	GM Dashboard refreshes	Manager sees live order count and team targets
11:00 PM	Last orders accepted	BD team stops taking orders after this
11:59 PM	AUTO: PO generated	System creates Purchase Orders for all 3 hubs
12:00 AM	PO reaches Purchase team	Shift user at each hub gets their PO
Morning	Vendor procurement	Executive buys items, boxes are labeled with barcodes
Afternoon	Hub receiving	Hub manager scans boxes, inventory updated

Afternoon	Delivery packed	Each pack barcoded, dispatched to customers
Evening	Delivery confirmed	Orders closed, bill auto-generated
Next Day	Bill approval chain	GM → Admin → CEO approves vendor payment
Repeats	Every single day	Fully automatic — no manual triggers needed

The Four Order Channels

How orders come in from Website, App, WhatsApp, and Manual entry — and how they all connect to the same ERP.

CHANNEL 1 — Website Orders

How does a website order reach the ERP?

When a customer clicks "Place Order" on your website, the website does not just save the order locally. It sends a message to the ERP using something called a Webhook. A webhook is like a phone call from the website to the ERP saying "Hey, someone just ordered 2kg of tomatoes." The ERP receives this call, reads the order details, and saves it in its database automatically.

■ **SIMPLE ANALOGY — What is a Webhook?**

Think of a webhook like a doorbell. When someone presses the bell (places an order), your house (ERP) is notified immediately. You do not have to keep checking outside every 5 minutes — the bell tells you on its own.

Step-by-Step: Website to ERP

1	Customer visits website and adds items to cart. • Website is built on any platform (Shopify, custom React, etc.) • Customer fills address, selects items, confirms order
2	Customer clicks "Place Order". • Website sends order data to ERP using a Webhook (HTTP POST request) • Data includes: customer name, address, items, quantities, total amount
3	ERP receives the webhook. • A Supabase Edge Function is always listening for incoming webhooks • It validates the data — checks if customer exists, items are valid • If valid, it saves the order to the "orders" table in the database
4	Order appears in Central Order Queue. • Tagged as Source = "Website" • Hub assignment engine runs automatically (see Part 4) • BD team can see this order in real time on their dashboard

CHANNEL 2 — Mobile App Orders

How does the mobile app connect to the ERP?

The mobile app is built using React Native (same code works for iPhone and Android). It is directly connected to Supabase — the same database that powers the ERP. When a customer places an order in the app, it writes directly to the ERP database in real time. There is no separate step — the app and ERP share the same data.

■ SIMPLE ANALOGY — How the App Connects

The mobile app and the ERP are like two windows of the same house looking at the same room. When someone puts something in the room from one window (app order), the other window (ERP dashboard) sees it immediately. They share the same space.

1

Customer opens the IGO mobile app and logs in.

- App connects to Supabase — the shared database
- Customer sees product catalogue, prices, and their order history

2

Customer selects items and places order.

- App writes order directly to the "orders" table in Supabase
- Order gets a unique ID, timestamp, customer ID, and item list

3

ERP dashboard updates in real time.

- BD team sees the new order appear without refreshing the page
- Uses Supabase Realtime — like a live feed, like WhatsApp messages appearing instantly

4

Hub is assigned and order enters the queue.

- Same hub assignment logic runs (geo-distance calculation)
- Order tagged Source = "Mobile App"

CHANNEL 3 — WhatsApp Orders (API + Manual)

How does WhatsApp connect to the ERP?

WhatsApp is the hardest channel to automate because customers type messages in free language. We use two approaches together: Automatic (Meta WhatsApp Business API) and Manual (staff reads and enters). This gives us the best of both worlds.

Option A: Automatic — Meta WhatsApp Business API

Meta (the company that owns WhatsApp) provides a Business API. This means your business gets an official WhatsApp number that can send and receive messages automatically, and those messages are delivered to your server as data.

■ SIMPLE ANALOGY — What is the WhatsApp Business API?

Normal WhatsApp is like a personal phone — you read messages with your eyes and type replies yourself. The WhatsApp Business API is like giving your ERP its own WhatsApp phone number that a computer can read and reply to. The computer reads each message and figures out what was ordered automatically.

1

Apply for WhatsApp Business API through Meta.

- Go to Meta for Developers (developers.facebook.com)
- Create a WhatsApp Business account, verify your business phone number
- Meta gives you an API token — a secret password the ERP uses to connect

2

Set up a Webhook on the ERP to receive WhatsApp messages.

- Every WhatsApp message sent to your business number arrives at the ERP
- A Supabase Edge Function receives the message text as raw data
- Example: Customer texts "2kg tomato 1kg onion" — ERP receives this text

3

Message Parser reads the text and extracts the order.

- A simple AI parser (using keywords) reads the message
- It recognises item names and quantities from natural language
- Example: "2kg tomato" → item=Tomato, qty=2, unit=kg

4

Parsed order is saved to the ERP order queue.

- Tagged as Source = "WhatsApp"
- Customer phone number is linked to their customer profile
- If parser is unsure, it flags the order for manual review

5

ERP sends an automatic WhatsApp confirmation back to customer.

- Customer gets a reply: "Your order has been received. Hub: Hub 1. Delivery tomorrow."
- This reply is sent automatically by the ERP via the same API

Option B: Manual Fallback — Staff Reads and Enters

For any message the parser cannot understand, or for customers who call on phone, the BD team member reads the message and enters the order manually into the ERP using a simple form. This ensures no order is ever missed.

- The Manual Entry form in the ERP has a "Source" field where the staff member selects "WhatsApp" or "Phone Call" so every order is properly tracked even if it was entered by hand.

CHANNEL 4 — Manual ERP Entry

Direct entry by BD team for any order not from digital channels.

1

BD team member opens ERP on laptop or phone.

- They go to the New Order form in the Sales module
- They search for the customer by name or phone number

2

Fill in order details.

- Select items from the product catalogue
- Enter quantities, delivery date, special instructions
- Select Source: Phone / Walk-in / Field Visit / WhatsApp (manual)

3

Submit — order enters the central queue.

- Same hub assignment logic runs
- Order creator ID is saved — so management knows who took this order
- Order appears in the BD team member's personal target counter

Central Order Hub

How all 4 channels land in one single place — one database, one view, zero duplication.

Every order — no matter where it came from — ends up in one database table called "orders". This is the Central Order Hub. It is like a single inbox that receives mail from email, WhatsApp, post, and hand delivery — all in one place.

■ **SIMPLE ANALOGY — Central Order Hub**

Imagine 4 different roads all leading to the same market. Whether a customer comes by bus, car, bicycle, or foot, they all arrive at the same market. The ERP order queue is that market — it does not matter which road the order took to get there.

What Data Is Stored For Every Order?

Database Field	What It Means	How It Is Set
order_id	A unique code for every order, e.g. ORD-2024-00045	Auto-generated
customer_id	Links to the customer's profile (name, address, phone)	Required
source	Where the order came from: Website / App / WhatsApp / Manual	Required
created_by	Which BD team member created or received the order	Required
hub_id	Which hub will fulfil this order (auto-assigned)	Auto-assigned
items	List of items ordered with quantities and units	Required
total_amount	Total order value in rupees	Calculated
status	Current state: Pending / Confirmed / Dispatched / Delivered	Updated
created_at	Exact date and time the order was placed	Auto-timestamp
delivery_date	When the customer expects delivery	Required
notes	Any special instructions from customer or BD team	Optional

Duplicate Detection — How We Prevent Double Orders

Sometimes a customer calls AND sends a WhatsApp message for the same order. Without a check, this would create two orders. The ERP has a duplicate detection system that looks for: same customer + same items + same date + different source. If found, it alerts the BD team to review before confirming.

■ Duplicate orders = double procurement = wasted money. The duplicate check runs automatically every time a new order is created. The team gets an alert and must confirm or merge before the order is finalised.

Hub Assignment Engine

How the system automatically picks the right hub for every customer order based on their delivery address.

You have 3 hubs. Every customer order must be assigned to the hub closest to the delivery address. Doing this manually for hundreds of orders a day would be impossible. The Hub Assignment Engine does it automatically in milliseconds.

■ SIMPLE ANALOGY — Hub Assignment Engine

Think of it like food delivery apps (Swiggy/Zomato). When you order food, the app automatically picks the restaurant nearest to you. You do not choose which branch — the app calculates the distance and picks the best one. Our hub engine does the exact same thing for every customer order.

Step-by-Step: How Hub Assignment Works

1

Customer address is captured with the order.

- Example: "123 Main Road, Banjara Hills, Hyderabad 500034"
- The address is stored exactly as entered

2

The address is Geocoded — converted to GPS coordinates.

- We send the address text to a Geocoding API (like Google Maps API)
- The API returns: Latitude = 17.4156, Longitude = 78.4347
- These coordinates are the precise location on earth's map

3

Distance to each hub is calculated using the Haversine formula.

- Haversine formula = a maths formula that calculates distance between two GPS points
- Distance to Hub 1 = 3.2 km | Hub 2 = 7.8 km | Hub 3 = 12.4 km
- This takes less than 1 second to calculate

4

The nearest hub is automatically assigned.

- Hub 1 wins (3.2 km is shortest) — order is tagged hub_id = Hub 1
- This happens the moment the order is created — no human decision needed

5

Manual override is available.

- BD team sees the assigned hub and can change it if needed
- Example: Customer is near Hub 1 but wants delivery from Hub 2 (stock issue)
- Every manual change is logged with the reason and who made the change

■ The geocoding API call costs a tiny fraction of a rupee per address. For 500 orders a day, this is practically free. We cache addresses so repeat customers do not trigger a new geocoding call.

FF GM Command Dashboard

What the General Manager sees and controls — targets, orders, PO approval.

The FF GM Dashboard is the manager's cockpit. Every important number about the day's orders, the team's performance, and the pending Purchase Orders is visible here. The GM does not need to ask anyone — the dashboard tells the full story.

What the Dashboard Shows

Team Target Cards	Each BD team member has a card showing: Daily Target (e.g. 50 orders) · Orders Taken Today (e.g. 32) · Remaining (18) · % Completed (64%). The GM can see at a glance who is on track and who needs a nudge.
Live Order Feed	A table of all orders placed today, refreshed every hour. Each row shows: Order ID · Customer Name · Items · Amount · Hub · Source · Created By. The GM can click any order to see full details.
Hub-wise Summary	Three boxes side by side — Hub 1, Hub 2, Hub 3 — each showing total orders and total value for that hub today. This helps the GM see which hub has the most demand.
PO Preview Panel	Before 11:59 PM, the GM can see a preview of what the Purchase Order will look like for each hub: Item Name · Current Stock · Orders Needed · PO Quantity. This is read-only — just to review before approving.
Early Approval Button	A single button per hub: "Approve and Generate PO Now". If the GM clicks this before 11:59 PM, the PO is generated immediately. If not clicked, the system auto-generates at exactly 11:59 PM.

EOD Purchase Order Engine

How Purchase Orders are automatically generated at exactly 11:59 PM every night — with no one pressing a button.

■ SIMPLE ANALOGY — The EOD PO Engine

Imagine a very organised assistant who, at exactly midnight every night, looks at all today's orders, checks what is in the warehouse, calculates what needs to be bought tomorrow, writes up the shopping list (Purchase Order), and sends it to the right buyer — all while everyone is sleeping. That is the EOD PO Engine.

What is a Cron Job?

A Cron Job is a task that runs automatically at a specific time. Like setting an alarm on your phone — except instead of waking you up, it runs a program on the server. We set a Cron Job to run at 23:59 (11:59 PM) every day. It cannot be forgotten, delayed, or skipped.

Step-by-Step: What Happens at 11:59 PM

1

Cron Job fires at exactly 23:59.

- A Supabase Scheduled Edge Function is triggered automatically
- No human action required — it runs every night without fail
- It begins processing each hub one by one: Hub 1, Hub 2, Hub 3

2

System reads all confirmed orders for today, grouped by hub.

- Query: "Give me all orders from today for Hub 1, item by item"
- Result: Tomato: 45kg · Onion: 30kg · Potato: 60kg (example)
- Same done for Hub 2 and Hub 3 separately

3

System reads current inventory for each hub.

- Query: "How much stock does Hub 1 currently have?"
- Result: Tomato: 12kg · Onion: 8kg · Potato: 25kg (example)

4

PO Quantity is calculated: Orders Needed – Current Stock.

- Tomato: Need 45kg – Have 12kg = ORDER 33kg
- Onion: Need 30kg – Have 8kg = ORDER 22kg
- Potato: Need 60kg – Have 25kg = ORDER 35kg
- If stock already covers the order, PO quantity = 0 (no need to buy)

5

Purchase Order is created for each hub.

- One PO document per hub with the calculated quantities
- PO is stamped with: Hub ID · Date · Generated By = SYSTEM · Status = Pending

6

PO is pushed to the Purchase Executive queue.

- The PO appears in the purchase team's dashboard immediately
- A push notification is sent to the shift user logged into that hub
- PO is now locked — quantities cannot be changed (only amendment request)

Early Approval Path

■ If the GM clicks the Approve button before 11:59 PM, the exact same steps above run immediately at that moment. The 11:59 PM cron job then sees the PO already exists and skips that hub. Early approval = same result, just earlier.

Purchase Executive Portal

How the PO reaches the right person based on who is working the shift at each hub.

You have 3 hubs. Each hub has different staff working different shifts. Hub 1's PO should go to whoever is logged in and working at Hub 1 — not to someone at Hub 2. The system checks this automatically.

■ **SIMPLE ANALOGY — Shift-Based PO Routing**

Think of it like a delivery app assigning an order to the nearest delivery driver who is currently online. The system checks: which driver is active near this restaurant? Our system checks: which purchase executive is logged into this hub right now?

1

At 12:00 AM, the PO Router runs.

- The system has the POs ready (created at 11:59 PM)
- It now needs to find who to send each PO to

2

System checks which shift users are currently logged in per hub.

- The Shift Login system records: User ID · Hub ID · Login Time · Status = Active
- Query: "Who is logged into Hub 1 right now with role = purchase_executive?"
- If found: PO for Hub 1 → that person's queue

3

PO appears in the Purchase Executive's morning queue.

- When they open the ERP at the start of their shift, the PO is waiting
- They see: Hub · Item List · Quantities · Vendor Contact · Priority Status

4

Executive goes to vendor and procures items.

- They use the PO as their shopping list at the vendor's place
- As they buy each item, they mark it as "Procured" in the ERP on their phone
- Partial procurement is allowed — remaining items carry forward

5

On completion, executive marks PO as Fulfilled.

- This triggers the Barcode Generation process (see Part 8)
- Each box of goods gets its own unique barcode printed
- Goods are transported to the assigned hub

Barcode & QR Code Generation

What a barcode is, what goes inside it, how the ERP generates it, how it is printed, and how it is scanned — every detail.

What is a Barcode / QR Code?

A barcode is a pattern of lines (or squares for QR code) that represents data. When you scan it with a camera or scanner, it instantly shows the data that was encoded inside it. The barcode does not "store" data itself — it is like a key that unlocks data from the ERP database.

■ SIMPLE ANALOGY — What is a Barcode?

Think of a barcode as a unique name tag for a box. Instead of writing all the details on the box in handwriting (which can be misread or smudged), we print a pattern that any phone camera can read instantly and perfectly. The pattern is linked to all the information stored in the ERP — so scanning it shows everything automatically.

What Data Goes Inside Each Barcode?

Data Field	What It Means	Example Value
BOX_ID	Unique ID for this specific box	e.g. BOX-HUB1-PO045-001
item_code	The item inside this box	e.g. ITEM-TOMATO-001
item_name	Human-readable item name	e.g. Fresh Tomatoes
quantity	How many units in this box	e.g. 10 (kg or pieces)
unit	The measurement unit	e.g. kg / pcs / dozen
hub_id	Which hub this box is going to	e.g. HUB-001
po_reference	The Purchase Order this box belongs to	e.g. PO-2024-05-22-H1
vendor_id	Which vendor supplied this box	e.g. VEND-045
packed_date	When this box was packed	e.g. 2024-05-22
batch_number	Groups boxes from the same purchase together	e.g. BATCH-045-A

How the Barcode is Generated — Step by Step

- 1
- Purchase Executive marks items as procured and enters box details.**

 - For each box: enter Item Name, Quantity, Number of Boxes
 - Example: "Tomatoes · 10kg per box · 4 boxes" = 4 separate barcodes
 - The ERP creates a BOX_ID for each box automatically

2

The ERP generates the barcode/QR code using a code library.

- We use a library called "qrcode" (Python) or "jsbarcode" (JavaScript)
- The library takes the BOX_ID and converts it into a visual QR pattern
- This takes less than 1 second per barcode
- The barcode is saved as an image (PNG format) in the ERP's storage

3

A printable label PDF is generated.

- For each box, a label is created with:
- Top: Company logo + Hub Name
- Middle: Large QR Code + Barcode (both formats)
- Bottom: Item Name · Qty · PO Reference · Date — printed in plain text
- The label PDF is ready to download and print immediately

4

Executive prints the labels and sticks them on boxes.

- Labels are printed on any regular printer with sticker paper
- Each box gets its own label — no label = box cannot be received at hub
- QR code size should be minimum 3cm x 3cm to scan reliably

5

Boxes are transported to the hub.

- Driver takes labeled boxes to the assigned hub
- The barcode label on each box is the "passport" for entry at the hub

QR Code vs Barcode — Which to Use?

Feature	QR Code	Traditional Barcode
Scanner needed	Any phone camera	Phone camera or barcode gun
Data capacity	Large (up to 4000 chars)	Small (up to ~80 chars)
Scan angle	Works at any angle	Must be scanned straight-on
Damage tolerance	Works even if 30% damaged	Needs to be clean and straight
Our recommendation	✓ Use QR Code as primary	Use as backup on same label

■ Print BOTH a QR code and a barcode on every label. QR code for phone cameras, barcode for hardware scanners. This ensures the label works no matter what device the hub manager uses.

Hub Manager Scanner App

The React Native mobile app that hub managers use to receive boxes, run QC checks, and log wastage — all connected to ERP in real time.

Why a Separate App?

Hub managers are on the warehouse floor, not sitting at a desk. They need a phone app they can use with one hand while holding a box with the other. The app is simple, fast, and works even with slow internet. It is built in React Native so it works on both iPhone and Android from a single codebase.

Step-by-Step: Receiving a Delivery at the Hub

1	Hub Manager opens the Scanner App and logs in. • App asks: "Which Hub are you at?" → Select Hub 1 / Hub 2 / Hub 3 • Login is tied to their ERP employee account • App dashboard shows: Expected Deliveries Today · PO Reference numbers
2	Driver arrives with boxes. • Hub manager taps "Scan Incoming Box" • Phone camera opens — manager points it at the QR code on the box • App reads the code in under 1 second
3	App displays the box information. • Item: Fresh Tomatoes · Quantity: 10kg · PO: PO-2024-05-22-H1 • Hub Manager sees: Is this the right item? Is the quantity correct? • They have 3 options: ✓ Accept · ✗ Reject (QC Fail) · ■ Partial Accept
4	If Accepted — Inventory is automatically updated. • Stock for "Tomatoes" at "Hub 1" increases by 10kg immediately • ERP shows this increase in real time on the main dashboard • Box status changes from "In Transit" to "Received"
5	If Rejected (QC Fail) — Manager fills rejection details. • Reason codes: Damaged · Wrong Item · Quantity Mismatch · Poor Quality • Manager takes a photo of the rejected item using the app's camera • ERP logs the rejection and alerts the Purchase Executive and GM
6	Wastage Logging (separate step). • During or after receiving, manager can log wastage: items that are damaged, expired, or unusable • Enter: Item · Quantity · Reason (Damaged / Expired / Wrong delivery) • Stock is automatically reduced by the wastage amount • Wastage report goes to GM daily
7	All done — app shows summary. • "14 boxes scanned · 13 accepted · 1 rejected · 2kg wastage logged" • Hub Manager taps "Submit Receiving Report" • ERP records the complete session with timestamp and user ID

Offline Mode

■ The app has an offline queue. If the internet is slow or down, scans are saved locally on the phone. The moment internet returns, all saved scans sync to the ERP automatically. No data is ever lost due to connectivity issues.

Smart Inventory Management

How stock levels stay perfectly accurate across all 3 hubs at all times.

Inventory means knowing exactly how much of every item you have in stock at each hub right now. Every action that adds or removes stock triggers an automatic update. There is never a need to manually count and update stock.

Event That Changes Stock	Effect on Inventory	Example
Box arrives and is scanned	+ Add to stock	Hub 1 · Tomatoes +10kg
Delivery pack dispatched	– Remove from stock	Hub 1 · Tomatoes –2kg
QC rejection	– Remove from stock	Hub 1 · Tomatoes –1kg (rejected)
Wastage logged	– Remove from stock	Hub 1 · Tomatoes –0.5kg (damaged)
PO Engine runs	Reads current stock	Used to calculate PO quantities
Manual stock count	Correct any mismatch	Adjustment logged with reason

■ Because inventory updates on every scan, the EOD PO Engine always has accurate stock data when calculating purchase quantities. No guesswork, no overbuying.

Delivery & Distribution

How delivery packs are barcoded, dispatched, and confirmed — with inventory updated at every step.

1

Hub team packs items for each customer order.

- Each customer's items are packed into a delivery bag/box
- The ERP shows the packing list: Customer · Items · Quantities
- Packer marks each item as packed in the app

2

Each delivery pack gets its own barcode.

- ERP auto-generates a PACK barcode linked to the customer order ID
- Pack barcode contains: Customer ID · Order ID · Hub · Delivery Date
- Label printed and attached to the delivery pack

3

Driver scans each pack on dispatch.

- Driver opens the Scanner App, goes to "Dispatch"
- Scans each pack barcode before loading it on the vehicle
- On scan: inventory decrements · order status → "Out for Delivery"

4

Driver delivers to customer.

- Customer receives their order
- Driver marks delivery as "Delivered" in the app
- Customer order status updates to "Delivered" · timestamp recorded

5

Undelivered packs are returned.

- If customer is absent: pack returns to hub, stock is added back to inventory
- Return scan triggers: inventory + · order status → "Delivery Failed"
- BD team is notified to reschedule with customer

PO Bill & Approval Chain

How the vendor bill is auto-generated and flows through FF GM, Admin, and CEO for approval before payment.

After all deliveries are complete for the day, the ERP automatically generates a vendor bill from the Purchase Order. This bill then goes through a 3-step approval chain. Every approver gets a notification and must act within a set time.

Stage	Role	Action	Deadline
Auto-Generated	System	Bill created from PO data — items, quantities, rates, total	Instant
Step 1 — FF GM	Reviews	Checks quantities match what was received · Approves or flags	24 hrs
Step 2 — Admin	Validates	Verifies bill against procurement records · Confirms accuracy	24 hrs
Step 3 — CEO	Final	Reviews total amount · Gives final approval	48 hrs
Payment	Finance	Vendor payment issued · Vendor ledger updated · PO closed	Immediate

■ ■ If any approver does not act within their deadline, an escalation alert is automatically sent to the next level and to the Admin. No bill should ever get stuck in the approval chain.

Technology Stack — Explained Simply

Every tool we use, what it does, and why we chose it.

React (Web ERP) JavaScript library for building the web interface.	The ERP runs in your web browser. React makes it fast and interactive — like how Gmail updates without refreshing the page.
React Native (Mobile App) Framework for building iOS and Android apps from one codebase.	Write the code once, get an iPhone app AND an Android app. Used for the Hub Manager Scanner App and the Driver App.
Supabase (Database + Auth + Realtime) The backend — database, login system, and live updates.	Supabase is like the engine room of the ERP. It stores all data, handles logins, and pushes live updates to every dashboard.
PostgreSQL (Database) The actual database that stores all data.	Every order, every box scan, every inventory change, every employee record — all stored here. Supabase runs on PostgreSQL.
Edge Functions (Automation) Small programs that run on Supabase's servers.	Used for: receiving webhooks, generating POs at 11:59 PM, sending notifications, parsing WhatsApp messages. They run without a separate server.
Supabase Cron (Scheduler) Runs tasks automatically at set times.	This is how the 11:59 PM PO generation works. Set it once, it runs every night forever without anyone pressing a button.
WhatsApp Business API (Meta) Official API to send/receive WhatsApp messages programmatically.	Lets the ERP receive WhatsApp orders as data and send confirmation messages back to customers automatically.
Geocoding API Converts address text into GPS coordinates.	Used for hub assignment. We send "123 Main Road, Hyderabad" and get back GPS latitude/longitude coordinates to calculate distances.
QRCode / JSBarcode Library Generates QR codes and barcodes as images.	Takes a text string like "BOX-HUB1-PO045-001" and converts it into a scannable QR code image automatically.
Expo Camera (React Native) Barcode/QR scanner built into the mobile app.	Uses the phone's camera to scan barcodes. No extra hardware needed. Works with both QR codes and standard barcodes.
Push Notifications Alerts sent to phones/browsers instantly.	Used to notify Purchase Executives when a PO arrives, Hub Managers when a delivery is expected, and approvers when a bill needs review.
Vercel (Hosting) Where the web app is deployed and accessed online.	The ERP website runs on Vercel. It is accessible from any browser at igogroup.in. Vercel handles uptime and performance automatically.

Key Database Tables

The data structure that powers every part of the system.

A database table is like an Excel sheet — rows and columns. Every piece of information in the ERP lives in one of these tables. Here are the most important ones:

orders Every customer order from all channels	order_id, customer_id, hub_id, source, created_by, items (JSONB), status, created_at
order_items Individual line items within each order	id, order_id, item_id, quantity, unit, price
customers Customer profiles and addresses	id, name, phone, address, geo_lat, geo_lng, hub_id (cached)
hubs The 3 hub locations	id, name, address, geo_lat, geo_lng, radius_km, manager_id
purchase_orders Auto-generated POs per hub	id, hub_id, date, status, generated_by, approved_by, approved_at
po_items Items within each PO with quantities	id, po_id, item_id, order_qty, stock_qty, purchase_qty
inventory Current stock levels per item per hub	id, hub_id, item_id, quantity, unit, last_updated
inventory_log Every stock change ever made	id, hub_id, item_id, change_qty, reason, triggered_by, created_at
boxes Every physical box from vendor to hub	id, barcode_id, po_id, hub_id, item_id, quantity, status, scanned_at
delivery_packs Customer delivery packs	id, barcode_id, order_id, hub_id, status, dispatched_at, delivered_at
qc_records QC pass/fail records per box	id, box_id, result, reason, photo_url, checked_by, created_at
wastage_log Wastage records	id, hub_id, item_id, quantity, reason, photo_url, logged_by, created_at
po_bills Vendor bills generated from POs	id, po_id, vendor_id, amount, status, gm_approved_at, admin_approved_at, ceo_approved_at
shift_logs Who is logged into which hub and when	id, user_id, hub_id, role, login_at, logout_at, status
employee_targets Daily targets per BD team member	id, user_id, date, target_orders, achieved_orders, target_amount, achieved_amount

Implementation Phases

What to build first, what comes next, and how long each phase takes.

We do not build everything at once. We build in phases — each phase delivers working features that the team can use immediately while the next phase is built.

Phase 1 Foundation

Weeks 1–3

- Set up Supabase database with all key tables (orders, customers, hubs, inventory)
- Build the Central Order Queue — the single inbox for all orders
- Build the Manual Order Entry form in the ERP (Channel 4)
- Set up user roles and login system
- Deliverable: Team can manually enter and view orders in one place

Phase 2 Channel Connections

Weeks 4–6

- Connect Website via Webhook — website orders auto-appear in ERP
- Connect Mobile App via Supabase Realtime — app orders sync live
- Set up WhatsApp Business API with Meta — receive messages as data
- Build WhatsApp message parser — extract item names and quantities
- Build duplicate detection system
- Deliverable: All 4 channels feeding one order queue

Phase 3 Hub Assignment + GM Dashboard

Weeks 7–9

- Integrate Geocoding API — convert addresses to GPS coordinates
- Build distance calculation engine — assign nearest hub automatically
- Build manual override with audit trail
- Build FF GM Dashboard: team targets, hourly order feed, hub summaries
- Build PO Preview panel for GM
- Deliverable: Every order auto-routed to correct hub, GM has full visibility

Phase 4 EOD PO Engine + Purchase Portal

Weeks 10–12

- Build inventory management tables and stock tracking

- Build EOD PO Engine (11:59 PM cron job)
- Build PO quantity calculation logic (orders minus stock)
- Build shift-based PO routing system
- Build Purchase Executive dashboard and PO view
- Build GM early approval button
- Deliverable: POs generated automatically every night, routed to right person

Phase 5

Barcode System + Hub App

Weeks 13–16

- Build barcode/QR code generation engine in ERP
- Build printable label PDF generator
- Build React Native Hub Manager App (iOS + Android)
- Build barcode scanner in mobile app using Expo Camera
- Connect app scans to ERP inventory in real time
- Build QC check flow, rejection logging, and wastage logging
- Deliverable: Boxes tracked end-to-end from vendor to hub

Phase 6

Delivery + Billing + Approval

Weeks 17–19

- Build delivery pack barcode generation
- Build dispatch scan — updates inventory on send
- Build driver delivery confirmation flow
- Build PO bill auto-generation from fulfilled PO
- Build 3-step approval chain (GM → Admin → CEO)
- Build escalation alerts for missed approval deadlines
- Deliverable: Full end-to-end cycle complete and automated

Phase 7

Polish, Testing & Go-Live

Weeks 20–22

- Full system testing with real orders across all channels
- Performance testing — 500+ orders per day simulation
- Staff training: BD team, Hub Managers, Purchase Executives, GM
- Parallel run — old process + new ERP simultaneously for 2 weeks

→ Go-live: switch fully to new ERP system

→ Post-launch monitoring and bug fixes

■ Every part of this system is designed to work together as one automated machine. Once all 7 phases are live, the daily order-to-delivery cycle runs entirely on its own — from the moment a customer places an order to the moment the vendor is paid. The team focuses on growth. The ERP handles the rest.